

FDS 排序算法—考前速查

考点

比较排序下界：任意基于比较的排序最坏 $\Omega(N \log N)$ 。
桶排/基数排不基于比较，能突破下界。决策树证明： $N!$ 个叶子，深度至少 $\log N! = \Omega(N \log N)$ 。

算法	平均	最坏	额外空间	稳定?	比较?
插入 (Insertion)	$O(N^2)$	$O(N^2)$	$O(1)$	是	是
希尔 (Shell)	$O(N^{3/2})$	$O(N^2)$	$O(1)$	否	是
堆排 (Heap)	$O(N \log N)$	$O(N \log N)$	$O(1)$	否	是
归并 (Merge)	$O(N \log N)$	$O(N \log N)$	$O(N)$	是	是
快排 (Quick)	$O(N \log N)$	$O(N^2)$	$O(\log N)$	否	是
桶排 (Bucket)	$O(M + N)$	—	$O(M)$	是	否
基数 (Radix)	$O(P(N + B))$	$O(P(N + B))$	$O(N + B)$	是	否

1. 插入排序 (Insertion)

思想：像整理扑克牌，每次从后面取一张插入前面已排好的位置。

核心：不是交换，是右移 + 插入。

```
void insertionSort(int A[], int N) {
    int i, j, tmp;
    for (i = 1; i < N; i++) {
        tmp = A[i];
        for (j = i; j > 0 && A[j-1] > tmp; j--)
            A[j] = A[j-1]; // 比 tmp 大的右移
        A[j] = tmp; // 插入
    }
}
```

复杂度：最好 $O(N)$ (已有序)/最坏 $O(N^2)$ (逆序)。
逆序数：平均 $N(N-1)/4$ 。每次相邻交换消除恰好 1 个逆序。

易错点

tmp=A[i] 后从头开始找位置，而不是从 i 往前比。
for(int j=i;...) 声明的 j 在外面 A[j]=tmp 用不了。

2. 希尔排序 (Shellsort)

思想：不等距的插入排序。gap 从 $N/2$ 逐步缩到 1。

```
void shellSort(int A[], int N) {
    int gap, i, j, tmp;
    for (gap = N/2; gap > 0; gap /= 2)
        for (i = gap; i < N; i++) {
            tmp = A[i];
            for (j=i; j>=gap && A[j-gap]>tmp; j-=gap)
                A[j] = A[j-gap];
            A[j] = tmp;
        }
}
```

跟插入的区别 (只改 3 处)：(1) 外套 gap 循环 (2) j-1 → j-gap (3) j-- → j-=gap

Hibbard 增量： $h_k = 2^k - 1$ ，最坏 $\Theta(N^{3/2})$ 。
Sedgewick 最优： $\{1, 5, 19, 41, 109, \dots\}$ 。

易错点

while(gap!=1) 吞了 gap=1 这轮。i+=gap 只处理一组！
for(int j=i...) j 作用域只在 for 里，外面用不了。

3. 堆排序 (Heapsort)

思想：原地最大堆。根 (max) 和末尾交换，堆缩小，下滤，重复。

索引从 0 开始！left=2i+1, right=2i+2, parent=(i-1)/2。

```
void percdwn(int A[], int i, int N) {
    int child, tmp = A[i];
    while (2*i+1 < N) {
        child = 2*i + 1;
        if (child+1<N && A[child+1]>A[child]) child++;
        if (tmp >= A[child]) break;
        A[i] = A[child]; i = child;
    }
    A[i] = tmp;
}

void heapSort(int A[], int N) {
    for (int i = N/2-1; i >= 0; i--) // BuildMaxHeap O(N)
        percdwn(A, i, N);
    for (int i = N-1; i > 0; i--) { // 排序 O(N log N)
        int t=A[0]; A[0]=A[i]; A[i]=t;
        percdwn(A, 0, i);
    }
}
```

$T(N) = O(N \log N)$ ，原地，不稳定。

易错点

用最小堆做原地排序 (应该用最大堆)；忘了索引从 0 开始。j>1 漏了根；i<N 漏了最后一个元素。

4. 归并排序 (Mergesort)

思想：分治法。递归拆分，再合并两个有序段。

```
void merge(int A[], int l, int m, int r) {
    int i=l, j=m+1, k=0, n=r-l+1;
    int* T=(int*)malloc(n*sizeof(int));
    while(i<=m && j<=r)
        T[k++]=(A[i]<=A[j])?A[i++]:A[j++];
    while(i<=m) T[k++]=A[i++];
    while(j<=r) T[k++]=A[j++];
    for(int p=0;p<n;p++) A[l+p]=T[p];
    free(T);
}

void mergeSort(int A[], int l, int r) {
    if (l>=r) return;
    int m=(l+r)/2;
    mergeSort(A,l,m); mergeSort(A,m+1,r);
    merge(A,l,m,r);
}
```

$T(N) = 2T(N/2) + O(N) \rightarrow O(N \log N)$ 。需要额外 $O(N)$ 空间。稳定。

易错点

tmp 的 k 从 left 开始 (应为 0)；忘了 free；n 声明在 malloc 后面。

5. 快速排序 (Quicksort)

思想：选 pivot，小的放左，大的放右，递归。

最关键的考试排序！

```
int partition(int A[], int l, int r) {
    int pivot = A[r], i = l;
    for (int j=l; j<r; j++)
        if (A[j] < pivot) {
            int t=A[j]; A[j]=A[i]; A[i]=t; i++;
        }
    int t=A[i]; A[i]=A[r]; A[r]=t;
    return i;
}

void quickSort(int A[], int l, int r) {
    if (l>=r) return;
    int p=partition(A,l,r);
    quickSort(A,l,p-1); quickSort(A,p+1,r);
}
```

手算：[5,2,6,1,4,9,3,7], pivot=7。i 是小区域的边界，j 扫描。遇到比 7 小的就换到 i 处。最后 pivot 放 i 处。
结果：[5,2,6,1,4,3,7,9], pivot=7 位置 6。

Median-of-Three 优化：取 $A[l]$, $A[(l+r)/2]$, $A[r]$ 的中值作 pivot。

最坏：每次 pivot 最小/最大 $\rightarrow O(N^2)$ ；**平均：** $O(N \log N)$ 。
不稳定。

易错点

pivot 选 $A[0]$ 遇到已排序数组直接退化 $O(N^2)$ ；忘了 swap pivot 到中间。

6. 表排序 (Table Sort)

动机：大结构体移动代价高。只排索引表 T。

$A = [f, d, a, b, e, c]$

$T = [0, 1, 2, 3, 4, 5] \rightarrow$ 排序后 $T = [2, 3, 5, 1, 4, 0]$
 $T[0]=2 \rightarrow A[2]=a$ (最小)

物理重排 (置换环)：把 A 按 T 重排。每个环移动 (环长 +1) 次。

最坏 $\lfloor N/2 \rfloor$ 个环，共 $\lfloor 3N/2 \rfloor$ 次移动。 $T = O(mN)$, $m =$ 结构体大小。

```
void rearrange(ElementType A[], int T[], int N) {
    for (int i=0; i<N; i++)
        if (T[i]!=i) {
            ElementType tmp=A[i]; int j=i;
            while(T[j]!=i){
                A[j]=A[T[j]]; int next=T[j];
                T[j]=j; j=next;
            }
            A[j]=tmp; T[j]=j;
        }
}
```

7. 桶排序 (Bucket Sort)

思想：不比较，按值分类投放。每个桶就是一个计数器。

```
// 例：给 N 个 O-M 分的学生成绩排序
void bucketSort(int A[], int N, int M) {
    int* count = (int*)calloc(M+1, sizeof(int));
    for (int i=0; i<N; i++) count[A[i]]++;
    for (int i=0, j=0; i<=M; i++)
        while (count[i]-- > 0) A[j++] = i;
    free(count);
}
```

$T = O(M + N)$ 。M = 桶数。**不基于比较**，突破 $O(N \log N)$ 。稳定。

8. 基数排序 (Radix Sort — LSD)

思想：从最低位 (LSD) 到最高位 (MSD)，每轮按当前位入桶排序。

手算：[64, 8, 216, 512, 27, 729, 0, 1, 343, 125]

Pass1(个位): 0: [0] 1: [1] 2: [512] 3: [343] 4: [64] 5: [125]
6: [216] 7: [27] 8: [8] 9: [729]
 $\rightarrow 0, 1, 512, 343, 64, 125, 216, 27, 8, 729$

Pass2(十位): 0: [0, 1, 8] 1: [512, 216] 2: [125, 27, 729]
3: [343] 6: [64]
 $\rightarrow 0, 1, 8, 512, 216, 125, 27, 729, 343, 64$

Pass3(百位): 1: [125] 2: [216] 3: [343] 5: [512] 7: [729]
0: [0, 1, 8, 27, 64]
 $\rightarrow 0, 1, 8, 27, 64, 125, 216, 343, 512, 729$

```
// LSD Radix Sort, 假设 10 进制, maxDigits=最大位数
void radixSort(int A[], int N, int maxDigits) {
    int* bucket[10], cnt[10];
    for (int d=0; d<maxDigits; d++) {
        for (int i=0; i<N; i++) cnt[i]=0;
        for (int i=0; i<N; i++)
            cnt[(A[i]/exp)%10]++;
        // 前缀和 → 各桶起始位置
        for (int i=1; i<10; i++) cnt[i]+=cnt[i-1];
        // 从后往前收集 (保证稳定!)
        for (int i=N-1; i>=0; i--) {
            int d=(A[i]/exp)%10;
```

```
        tmp[--cnt[d]]=A[i];
    }
    for (int i=0;i<N;i++) A[i]=tmp[i];
    exp*=10;
}
}
```

$T = O(P \cdot (N + B))$, P= 位数, B= 基数 (10)。稳定, 不基于比较。

MSD vs LSD: MSD 从高到低 (分治), LSD 从低到高 (稳定)。

卡片排序: LSD 从最不重要到最重要字段依次排。

决策树与排序下界

考点

决策树证明 $\Omega(N \log N)$ 下界

- N 个元素有 $N!$ 种排列 $\rightarrow$ 决策树至少 $N!$ 个叶子
- 二叉树深度 k 最多 2^{k-1} 个叶子
- $N! \leq 2^{k-1} \rightarrow k \geq \log N! + 1 = \Omega(N \log N)$

3 个元素插入排序决策树:

$N = 3, N! = 6$, 深度 $k \geq 4$ (至少 4 次比较)。

稳定性总结

稳定: 插入、归并、桶、基数。

不稳定: 希尔、堆、快排。

口诀: 稳定排队 (插入归并桶基数), 不稳定快跑 (快希尔堆)。

Additional: Replacement Selection

外部排序的预处理: 用最小堆生成更长的初始顺串 (run)。

平均 run 长度 $= 2M$ (M= 堆大小), 比普通分块长一倍。